



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:**

« Компилятор базисного подмножества

Рефала-5 с представлением данных,

использующим раскрученные очереди »

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель

(Подпись, дата)

(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Обзор предметной области	5
1.1 Программа, функции и предложения	5
1.2 Объектные выражения, термы и символы	6
1.3 Неоднозначное сопоставление	7
2 Исследование и анализ решений	8
2.1 Объектное выражение как абстрактный тип	8
2.2 Классическое (плоское двусвязное) представление	9
2.3 Компромиссное представление с подвешенными скобками	10
2.4 Массивное представление	11
2.5 Векторно-списковое представление	12
2.6 Гипотетическое представление: дерево конкатенаций	13
2.7 Предлагаемое представление: раскрученная двусторонняя очередь	13
2.7.1 Структура мелкого и глубокого уровней	14
2.7.2 Краткое обоснование амортизированных оценок	15
2.7.3 Сводное сравнение представлений	16
2.7.4 Область применимости и ограничения	16
2.8 Выводы по разделу	17
3 Проектирование программной системы	18
3.1 Функциональные требования	18
3.2 Архитектура и потоки данных	18
3.3 Внутренние представления данных	19
3.4 Интерфейс контейнера выражений	19
3.5 Декомпозиция на модули	19
3.6 Модель выполнения и точки расширения	20
3.7 Нефункциональные требования	21
3.8 Обработка ошибок и форматы сообщений	21
3.9 Выбор и сокращение встроенной библиотеки	21
3.10 Представление термов и вложенность	21

3.11 Окружение сопоставления и представление подстановок . .	22
3.12 Контракт механизма сопоставления	23
3.13 Сценарии использования системы	23
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ВВЕДЕНИЕ

Язык Рефал-5 относится к семейству языков, основанных на переписывании термов: программа задаётся набором правил сопоставления образца с объектным выражением и построения результирующего выражения. На практике стоимость вычислений в таком подходе часто определяется не только числом применённых правил, но и эффективностью представления объектных выражений и операций над ними (вставка/удаление с концов, конкатенация).

Цель НИР: исследовать и обосновать применение раскрытых двусторонних очередей для представления объектных выражений при реализации базисного подмножества Рефала-5, а также выполнить проектирование программной системы, реализующей выбранное подмножество языка.

Для достижения цели решаются задачи:

- изучить базовые конструкции Рефала-5 и определить состав базисного подмножества, достаточного для практического использования;
- исследовать операции над объектными выражениями, определяющие вычислительную стоимость выполнения правил;
- описать раскрытую двустороннюю очередь (мелкий и глубокий уровни представления), её инварианты и принцип ребалансировки;
- сравнить предложенное представление с альтернативами и сформулировать область применимости;
- спроектировать архитектуру программной системы (входной язык, внутреннее представление, модули и интерфейсы).

1 Обзор предметной области

В разделе кратко описаны базовые конструкции и терминология языка Рефал-5, необходимые для дальнейшего обсуждения. Основными понятиями являются объектное выражение, сопоставление с образцом и построение результата правила. В качестве иллюстраций приведены несколько простых функций, которые будут использоваться в разделе 2 при разборе вычислительной стоимости разных представлений объектных выражений в памяти.

1.1 Программа, функции и предложения

Программа на Рефале — это набор функций. Каждая функция задаётся одним или несколькими **предложениями** (правилами) вида «образец = результат», заключёнными в фигурные скобки. При вызове функции интерпретатор просматривает её предложения сверху вниз и берёт первое, чей образец сопоставим с аргументом [1, 2]. По подстановке, полученной при сопоставлении, строится результирующее выражение, которое заменяет вызов в текущем поле зрения.

Простейший пример — рекурсивная функция, вычисляющая длину объектного выражения (листинг 1):

Листинг 1: Функция подсчёта длины выражения

```
1 Length {  
2     = 0;  
3     s.First e.Rest = <Add 1 <Length e.Rest>>;  
4 }
```

Функция `Length` имеет два предложения. Первое срабатывает на пустом выражении и возвращает 0. Второе сопоставляется с непустым выражением: оно связывает первый терм с переменной `s.First`, остаток — с переменной `e.Rest` и рекурсивно подсчитывает длину остатка. Имена с префиксами `s.`, `t.`, `e.` обозначают переменные образца разных типов; их семантика рассматривается ниже.

1.2 Объектные выражения, термы и символы

Все данные в Рефале представлены **объектными выражениями**. Объектное выражение — это конечная последовательность **термов** (возможно пустая). Каждый терм имеет один из двух видов:

- **атомарный терм** — собственно **символ** в терминологии Рефала; в Рефале-5 различают три разновидности символов: **символ-слово** (идентификатор: Found, True, Nil), **макроцифра** (целое число произвольной длины) и **литера** (один знак строкового литерала);
- **скобочный терм** — объектное выражение, заключённое в круглые скобки.

Важно не смешивать понятия «символ» и «литера». Символ-слово — это атомарный объект (метка), а не последовательность литер: при сопоставлении Found ведёт себя как один атомарный терм. Запись в одинарных кавычках, например 'abc', обозначает три литеры подряд, то есть объектное выражение длины три. Аналогично, число 42 — это один символ (макроцифра), а не последовательность из двух цифр-литер.

Сопоставление с образцом опирается на три класса **переменных образца**:

- **s-переменная** (*s.X*) связывается с одним символом — одним атомарным термом любой из трёх разновидностей;
- **t-переменная** (*t.X*) связывается с одним термом, в том числе со скобочным;
- **e-переменная** (*e.X*) связывается с произвольной, возможно пустой, цепочкой термов.

Простой пример с переменной *e*-типа — функция, возвращающая последний символ непустой последовательности атомарных термов (листинг 2):

Листинг 2: Функция получения последнего символа

```
1 Last {  
2   e.Init s.Last = s.Last;  
3 }
```

При вызове <Last 1 2 3> переменная *e.Init* связывается с выражением 1 2, а *s.Last* — с символом 3. Вариант, в котором *e.Init* забирала бы всё выражение целиком, не годится: тогда для *s.Last* не осталось бы ни одного терма, и сопо-

ставление было бы неудачным. Алгоритм сопоставления учитывает требования к концам образца и подбирает разбиение, при котором все переменные связываются согласованно.

1.3 Неоднозначное сопоставление

Если в образце встречается несколько *e*-переменных, сопоставление становится **неоднозначным**: одно и то же объектное выражение можно разбить разными способами. Стандартный пример — функция, проверяющая вхождение заданного символа в выражение (листинг 3):

Листинг 3: Неоднозначное сопоставление при поиске символа

```
1 Contains {  
2   s.Needle e.Left s.Needle e.Right = True;  
3   s.Needle e.Any                      = False;  
4 }
```

Первое предложение требует, чтобы и в начале, и где-то дальше в выражении встретился один и тот же символ `s.Needle`. Образец содержит две *e*-переменные `e.Left` и `e.Right`, и при сопоставлении интерпретатор перебирает варианты: он начинает с пустого `e.Left`, и если очередной `s.Needle` не оказывается на текущей позиции, удлиняет `e.Left` на один терм и пробует снова. Каждое такое удлинение — это комбинация операций «взять один терм с начала остатка» и «дописать этот терм в конец накапливаемого префикса», то есть отделение начала и конкатенация.

Иными словами, даже логически простое правило приводит к множеству шагов, на каждом из которых интерпретатор разбивает и собирает фрагменты выражения. Стоимость отдельного такого шага зависит от того, как выражение представлено в памяти. Этому вопросу посвящён раздел 2.

2 Исследование и анализ решений

В разделе рассматривается, как объектное выражение хранится и преобразуется в памяти при реализации Рефала-5. Сначала фиксируется набор базовых операций над выражением (2.1), который нужен интерпретатору при сопоставлении и построении результата. Далее обсуждаются распространённые представления объектных выражений и их практические компромиссы. Завершает раздел описание представления, предлагаемого в данной работе, и сводное сравнение с альтернативами.

2.1 Объектное выражение как абстрактный тип

Чтобы говорить о стоимости разных представлений, удобно зафиксировать набор операций, через которые выражается интерпретация любого предложения:

- **проверка на пустоту**;
- **отделение начала**: получить первый терм и остаток выражения;
- **отделение конца**: получить последний терм и остаток выражения;
- **конкатенация двух выражений** — основная операция при построении результата правила;
- **выражение из одного термина** (создание сингулярного выражения).

Эти операции вызываются на каждом шаге сопоставления и каждом шаге сборки результата. Например, при переборе e -переменной (1.3) последовательно работают отделение начала и конкатенация; при возврате результата правила вида $= \langle F \ s.X \rangle \langle G \ e.Y \rangle$; итоговое выражение собирается из нескольких фрагментов с помощью конкатенации. Поэтому выбор представления — это, по существу, выбор того, какие из перечисленных операций должны быть дешёвыми, а какие могут стать дорогими.

Ниже сначала рассматриваются два классических списковых представления, в которых дешёвы операции на концах и конкатенация, а проблемой становится копирование значений переменных. Затем разбираются массивные и векторно-списковые представления, в которых, наоборот, копирование дешёво, а конкатенация может становиться дорогой. После этого описано гипотетическое представление на основе деревьев конкатенаций. В конце предлагается раскрученная двусторонняя очередь, амортизированно устраняющая обе проблемы.

2.2 Классическое (плоское двусвязное) представление

Этот способ представления использовался в большинстве реализаций Рефала вплоть до 1980-х годов и часто называется **классическим** [3] или **плоским двусвязным** [4]. Объектное выражение хранится как двусвязный список звеньев, где каждое звено соответствует либо атомарному символу, либо одной из двух структурных скобок. У звена-скобки есть дополнительное поле, указывающее на парную скобку. Контейнер выражения хранит указатели на первое и последнее звено.

Стоимость основных операций при таком представлении:

- отделение начала или конца — $O(1)$: сдвинуть указатель; если крайнее звено — открывающая скобка, можно прыгнуть через поле парной скобки на закрывающую и получить скобочный терм целиком;
- конкатенация двух выражений — $O(1)$: достаточно перенастроить четыре указателя на стыке;
- копирование выражения — $O(N)$, где N — число звеньев в записи (включая все вложенные скобки): необходимо физически создать новый список звеньев.

Дешёвая конкатенация делает классическое представление удобным для построения результата, однако дорогое копирование порождает классическую **проблему копирования**: если переменная встречается в правой части предложения большее число раз, чем в левой, каждое «лишнее» вхождение требует полной копии её значения. Стандартная иллюстрация — функция подстановки из препринта Абрамова и Романенко [4]: для каждого символа выражения по таблице ищется его замена.

Листинг 4: Подстановка по таблице (пример проблемы копирования)

```
1 Subst {  
2   (e.Tab)           = ;  
3   (e.Tab) s.X e.Rest = <Lookup (e.Tab) s.X> <Subst (e.Tab)  
   e.Rest>;  
4 }
```

Переменная `e.Tab` в левой части появляется один раз, а в правой — дважды (передаётся в `Lookup` и в рекурсивный вызов `Subst`). При классическом представлении это означает, что на каждом шаге рекурсии таблица копируется целиком. Если длина исходного выражения и длина таблицы оба порядка n , общая стоимость становится $O(n^2)$, и попытка программировать в функциональном стиле упирается в эту квадратичную асимптоту. Распространённый «инженерный» обход — так называемый «сквозной просмотр выражения» с двумя стеками, переводящий `Subst` в императивный по духу вариант без многократного использования `e.Tab`; такое решение позволяет избежать копирований, но затемняет код и препятствует распараллеливанию [4].

2.3 Компромиссное представление с подвешенными скобками

Чтобы смягчить проблему копирования, не отказываясь от списковой основы, было предложено **компромиссное представление** [3]. Оно используется в реализации Рефала-6 и ряде учебных проектов, а также обсуждается в документации Рефала-05 [2].

В этом представлении звено списка соответствует не отдельному знаку записи, а одному **терму верхнего уровня**. Скобочный терм хранится особым образом: вместо двух парных звеньев (открывающей и закрывающей скобок) звено содержит ссылку на отдельный кольцевой список, представляющий вложенное выражение. У головы кольца хранится **счётчик ссылок**.

Стоимость операций по сравнению с плоским списком:

- отделение начала и конца — $O(1)$, как и раньше;
- конкатенация — $O(1)$;
- копирование верхнего уровня — $O(N_{\text{верх}})$, где $N_{\text{верх}}$ — число термов верхнего уровня; вложенные выражения при копировании не дублируются, у их кольцевых списков лишь увеличивается счётчик ссылок.

Применительно к функции `Subst` это даёт ощутимый выигрыш в типичных сценариях: если таблица состоит из пар `(s.Key e.Value)`, то при копировании `e.Tab` физически дублируются только звенья верхнего уровня, а вложенные пары остаются разделёнными между копиями.

Платой становится более сложное управление памятью: необходимо поддерживать счётчики ссылок при разрушении и сборке выражений и аккуратно обрабатывать ситуации, когда переменная связана со внутренностью скобочного терма, который оказался расшарен (тогда модификация одной «копии» могла бы повредить другие). Кроме того, наличие циклических связей у голов колец усложняет автоматическую сборку мусора.

2.4 Массивное представление

Альтернативный путь, предложенный в работе Абрамова и Романенко [4], состоит в том, чтобы хранить объектное выражение в **непрерывном участке памяти** (массиве тегированных ячеек). Каждая ячейка содержит либо атомарный символ, либо открывающую или закрывающую структурную скобку. Скобочный терм при таком хранении не требует отдельного указателя: он занимает диапазон ячеек от открывающей до закрывающей скобки.

Главное преимущество массивного представления — **дешёвое копирование переменной**. Значение e -переменной $e.X$ представляется парой индексов (l, r) внутри массива; копирование переменной — это просто копирование пары чисел, то есть $O(1)$. Это превращает функциональный стиль программирования в Рефале из «дорогого» в «нормальный»: даже многократное использование одной и той же переменной в правой части не приводит к физическому дублированию данных. На функции Subst из 2.2 массивное представление даёт линейную асимптотику без каких-либо ухищрений с императивным переписыванием.

Стоимость основных операций:

- отделение начала и конца — $O(1)$: сдвинуть один из индексов;
- копирование переменной — $O(1)$;
- конкатенация двух выражений длины $|X|$ и $|Y|$ — $O(|X| + |Y|)$: требуется выделить новый массив подходящей длины и физически скопировать в него содержимое обоих фрагментов.

Так возникает **проблема конкатенации**, симметричная проблеме копирования. Если результат правила собирается в цикле, как в типовой функции отображения

Листинг 5: Построение результата через рекурсивную конкатенацию

```
1 Map {  
2     = ;  
3     s.X e.Rest = <F s.X> <Map e.Rest>;  
4 }
```

то на n шагах рекурсии возникают n конкатенаций. При наивной массивной реализации каждая из них копирует уже накопленный префикс, и суммарная стоимость становится $O(n^2)$.

В препринте [4] предлагается смягчение — так называемые **дырки** (запас свободного места) по краям массива. Если к выражению с дыркой справа дописывается короткий фрагмент длины k , то вместо создания нового массива достаточно скопировать только эти k ячеек в свободную область. На сериях коротких приписываний это даёт амортизированно линейное общее время. Однако если дырка исчерпывается, приходится снова выделять новый массив и переносить всё содержимое; гарантию $O(1)$ на произвольных последовательностях операций дырки не дают. Дополнительно в [4] обсуждаются меры по снижению затрат на конкатенацию — распознавание частных случаев, функции с несколькими входами и выходами, использование скобок для структуризации данных.

2.5 Векторно-списковое представление

Векторно-списковое представление, предложенное Скоробогатовым [3], нацелено на то, чтобы сохранить дешёвое копирование переменных от массивного представления и одновременно избежать линейной конкатенации.

Идея состоит в том, чтобы изменить интерфейс между функциями. Аргументы функций передаются как **векторы** (массивы тегированных ячеек), но **результат** функция возвращает не как один сплошной вектор, а как **список ссылок на фрагменты векторов**. Конкатенация фрагментов откладывается: пока вычисление не дошло до точки, где результат должен стать пассивным аргументом другой функции (в частности, попасть внутрь скобочного термина), фрагменты остаются отдельными звеньями.

Применительно к функции Map это означает следующее. На каждом шаге рекурсии вместо физического присоединения $<F\ s.X>$ к результату $<Map\ e.Rest>$ формируется лёгкое звено «сначала такой-то фрагмент, потом такой-то». Когда

вычисление функции завершено и её результат должен стать пассивным выражением, накопленный список фрагментов собирается в единый вектор за один проход, что даёт линейное общее время вместо квадратичного.

Платой становится более сложный интерфейс языка сборки и более ответственное управление временем жизни фрагментов: пока на фрагмент есть ссылки в незавершённых вычислениях, исходный вектор нельзя освободить или переиспользовать. Подробное описание представления и связанной системы команд абстрактной Рефал-машины приведено в [3].

2.6 Гипотетическое представление: дерево конкатенаций

Среди представлений, не получивших распространения в реализациях Рефала, интерес представляет **дерево конкатенаций** (gore). В этом представлении объектное выражение хранится в виде двоичного дерева, где листья — небольшие фрагменты массива, а внутренние узлы соответствуют операциям конкатенации поддеревьев.

Преимущество gore в том, что конкатенация двух выражений сводится к созданию одного нового внутреннего узла, что даёт $O(1)$ с периодической переконструкцией за $O(\log n)$. Копирование переменной также $O(1)$ — это ссылка на корень поддерева. Слабая сторона — **операции на концах**: чтобы получить первый или последний терм, нужно спуститься по дереву к крайнему листу, что стоит $O(\log n)$ и зависит от баланса. Для Рефала это особенно неудобно, потому что e -переменные требуют как раз многократного отделения термов с краёв.

В литературе по реализациям Рефала gore обсуждается лишь как теоретическая возможность [2].

2.7 Предлагаемое представление: раскрученная двусторонняя очередь

Все рассмотренные выше представления имеют выраженное «слабое место»: плоский список — проблему копирования; компромиссный — неполное её устранение и сложное управление памятью; массивное — проблему конкатенации;

векторно-списковое — сложный интерфейс и привязку к определённой модели вычисления; горе — дорогие операции на концах. В данной работе предлагается использовать **раскрученную двустороннюю очередь**, описанную в монографии Окасаки [5], как структуру, амортизированно решающую обе ключевые проблемы одновременно.

Идея структуры опирается на два уровня вложенности.

- **Мелкий уровень** (*Shallow*) — буфер фиксированной ёмкости B , организованный как циклический массив. Локальные операции на концах при наличии места выполняются как сдвиг индексов, без выделения памяти.
- **Глубокий уровень** (*Deep*) — тройка $\langle front, mid, back \rangle$, где крайние компоненты ($front$, $back$) — мелкие буферы, а средний (mid) — двусторонняя очередь мелких буферов. Структура определена рекурсивно: средняя очередь сама может быть как мелкой, так и глубокой.

Ключевой инвариант: внутренние буферы средней очереди заполнены не менее чем наполовину. Этот инвариант предотвращает вырождение глубокой структуры в длинную цепочку почти пустых буферов и обеспечивает, что глубина рекурсивного представления растёт логарифмически от длины выражения.

Стоимость основных операций:

- отделение начала или конца — $O(1)$ амортизированно;
- конкатенация двух выражений — $O(1)$ амортизированно (за счёт перетекания крайних буферов в среднюю очередь);
- использование переменной несколько раз — $O(1)$, благодаря структурному разделению: новая версия выражения ссылается на те же неизменяемые буферы и узлы, что и старая.

Структурное разделение в данном случае — инженерная техника, а не свойство языка: в самой модели Рефала-5 нет изменяемых ячеек памяти, и вычисление концептуально строит новые выражения, а задача реализации в том, чтобы за этой моделью не стояло физическое копирование.

2.7.1 Структура мелкого и глубокого уровней

Рисунок 1 показывает идею мелкого буфера: элементы размещены циклически, концы задаются индексами «head» и «tail»; в общем случае head может находиться правее tail, и разрыв «обходится» по модулю ёмкости B .

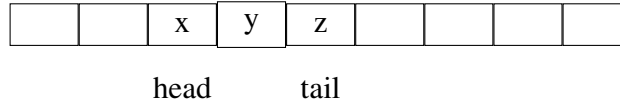


Рисунок 1 — Идея мелкого буфера: элементы размещены циклически, концы задаются индексами

Глубокий уровень состоит из двух мелких буферов и средней очереди, которая сама хранит мелкие буферы (рисунок 2). Рекурсивность означает, что средняя очередь при необходимости тоже становится глубокой, и мы получаем структуру с логарифмической по основанию $B/2$ глубиной.

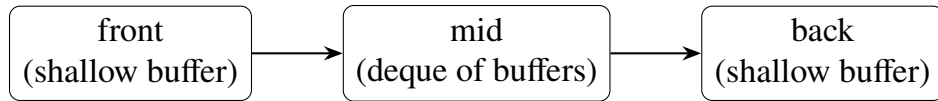


Рисунок 2 — Глубокий уровень раскрученной двусторонней очереди: два мелких буфера и средняя очередь буферов

2.7.2 Краткое обоснование амортизированных оценок

Большинство операций затрагивает только крайние буферы *front* и *back*: пока в них есть свободное место, добавление и удаление сводятся к сдвигу индексов в циклическом массиве и являются строго $O(1)$. Перетекание буфера в среднюю очередь происходит только при достижении границ заполненности и требует $O(B)$. Если каждой такой дорогой операции приписать стоимость, накопленную на предшествующих локальных операциях, то средняя стоимость шага остаётся около константы; это и есть классический амортизированный аргумент [5].

Интуитивно редкие глубокие переносы можно сравнить с цепочкой переносов при прибавлении единицы в позиционной системе счисления: вероятность цепочки длины k убывает геометрически, и вклад в среднюю стоимость образует сходящийся ряд. Для основания $q \geq 2$ ожидаемое число переносов

$$\mathbb{E}[K] = \sum_{k \geq 1} \frac{k(q-1)}{q^{k+1}} = \frac{1}{q-1}.$$

Это иллюстрация, а не строгое доказательство для конкретной реализации. Полные техники амортизированного анализа функциональных структур данных

(включая раскрутку и ленивые вычисления) систематически изложены у Окаски [5].

2.7.3 Сводное сравнение представлений

Сравнение рассмотренных представлений приведено в таблице 1. Зафиксирована стоимость трёх ключевых операций, через которые выражается основная вычислительная нагрузка интерпретатора: отделение термина с конца, конкатенация двух выражений и использование одного значения переменной несколько раз.

Таблица 1 — Сравнение представлений объектных выражений по стоимости основных операций

Представление	Концы	Конкатенация	Копирование переменной
Классическое (плоское двусвязное)	$O(1)$	$O(1)$	$O(N)$ (вся запись)
Компромиссное (подвешенные скобки)	$O(1)$	$O(1)$	$O(N_{\text{верх}})$
Массивное (с дырками)	$O(1)$	$O(X + Y)$, линейная амортизация при последовательных приписываниях	$O(1)$
Векторно-списковое	$O(1)$	откладывается, $O(1)$ на сборке	$O(1)$
Дерево конкатенаций (горе)	$O(\log n)$	$O(1)$, перебалансировка $O(\log n)$	$O(1)$
Раскрученная двусторонняя очередь	$O(1)$ амортиз.	$O(1)$ амортиз.	$O(1)$

2.7.4 Область применимости и ограничения

Раскрученная двусторонняя очередь оправдана прежде всего тогда, когда выражения достаточно длинные, а в правилах активно используются e -переменные, порождающие многократные разбиения и сборки. На коротких выражениях и в программах, где почти каждое правило связано с одним-двумя терминами, выигрыш от сложной структуры может уступить простым спискам или массивам из-за более высокого постоянного множителя.

Целевой реализацией в данной работе является среда JVM. Это естественно сочетается с неизменяемыми объектами и структурным разделением, но накладные расходы на работу сборщика мусора нужно учитывать при выборе ёмкости B мелкого буфера: слишком малое B приводит к большому числу мелких объектов, слишком большое — ухудшает локальность модификаций функционального стиля.

2.8 Выводы по разделу

Требования к представлению объектных выражений базисного подмножества Рефала-5 сводятся к следующему: дешёвые операции на концах, дешёвая конкатенация при построении результата и отсутствие физического копирования при многократном использовании одной переменной. Существующие представления закрывают часть этих требований: классическое и компромиссное списковые — быстрые на концах и при конкатенации, но дорогие на копировании; массивное и векторно-списковое — быстрые на копировании, но имеют ограничения по конкатенации или по интерфейсу языка сборки; горе — быстрая конкатенация и копирование, но дорогие операции на концах. Раскрученная двусторонняя очередь покрывает все три требования амортизированно за $O(1)$ и поэтому выбрана в качестве представления объектных выражений в проектируемой системе.

3 Проектирование программной системы

В третьем разделе фиксируется архитектура программной системы: состав модулей, интерфейсы и данные, которые они обрабатывают. Проектирование опирается на выбранное представление выражений (раскрученная двусторонняя очередь) и на грамматику базисного подмножества.

3.1 Функциональные требования

К системе предъявляются требования:

- приём исходного текста на подмножестве Рефала-5;
- выдача сообщений об ошибках с привязкой к позиции во входном тексте;
- выполнение программы в модели «выбор первого применимого правила» для функций;
- сокращённая библиотека встроенных функций (только необходимый минимум по постановке).

3.2 Архитектура и потоки данных

Система включает цепочку этапов: подготовка входа (при необходимости), лексический анализ, синтаксический анализ, семантический анализ и выполнение правил. Представление объектных выражений едино для этапов выполнения и используется при сопоставлении и сборке результата.

На практике удобно разрешить внешний «рассахариватель», который переводит исходный текст полного Рефала-5 к синтаксису базисного подмножества (условия, блоки и др.), чтобы не расширять грамматику минимальной системы. Такой шаг не меняет семантику рассматриваемых программ при условии корректности рассахаривания, но выносит сложные конструкции из ядра транслятора.



Рисунок 3 — Этапы обработки программы и основные данные

Целевая реализация рассматривается в экосистеме JVM (например, Java): это естественно сочетается с представлением программы и выражений как ссылочных структур и со сборкой мусора, но накладные расходы GC должны учитываться при выборе гранулярности объектов (термы, буферы раскрученной двусторонней очереди, узлы дерева программы).

3.3 Внутренние представления данных

В проектировании фиксируются следующие структуры:

- **токены** с координатами (строка/столбец) для точной диагностики;
- **дерево программы** как набор функций и правил сопоставления;
- **объектные выражения** как двусторонняя очередь термов.

3.4 Интерфейс контейнера выражений

Для независимости модулей удобно выделить интерфейс контейнера выражения, содержащий операции: проверку пустоты, отделение терма с начала/конца, добавление/удаление на концах, итерацию и конкатенацию. Конкретная реализация (мелкий и глубокий уровни раскрученной двусторонней очереди) скрыта за этим интерфейсом и может заменяться без изменения логики сопоставления.

3.5 Декомпозиция на модули

С практической точки зрения систему удобно декомпозировать на набор модулей со строго определёнными ответственностями:

- **модуль ввода** — чтение текста программы, выбор кодировки, единый входной поток символов;
- **подготовка входа** (опционально) — запуск внешнего рассахаривателя и получение текста базисного подмножества;
- **лексический анализатор** — преобразование потока символов в поток токенов с координатами;
- **синтаксический анализатор** — построение дерева программы (функции, предложения, выражения);

- **семантический анализатор** — проверки, которые удобнее выполнить до исполнения (например, определённость функций и согласованность переменных);
- **контейнер выражений** — представление объектных выражений и операции над ним;
- **механизм сопоставления** — сопоставление образца и выражения, вычисление подстановки;
- **исполнитель правил** — выбор первого применимого предложения и построение результата;
- **таблица встроенных функций** — набор примитивов и диспетчеризация вызовов.

Такая декомпозиция снижает связанность: развитие входного языка (например, уточнение синтаксиса) не затрагивает слой контейнера выражений, а оптимизация представления выражений не затрагивает логику синтаксического анализа.

3.6 Модель выполнения и точки расширения

Модель выполнения фиксируется как «вызов функции → просмотр предложений сверху вниз → первое успешное сопоставление». Отдельные точки расширения в проектировании:

- добавление новых классов термов (при необходимости расширить подмножество);
- расширение механизма сопоставления (например, введение дополнительных ограничений или условий);
- расширение библиотеки встроенных функций без изменения ядра сопоставления;
- возможное добавление оптимизаций (индексация предложений по первым термам, мемоизация вызовов).

Даже если оптимизации не входят в круг основных задач данного этапа, их перечисление как точек расширения полезно, поскольку показывает, что выбранный интерфейс контейнера выражений не препятствует дальнейшему развитию системы.

3.7 Нефункциональные требования

Помимо функциональности, для системы важны следующие свойства:

- **предсказуемость потребления памяти:** представление выражений должно использовать структурное разделение и избегать полного копирования;
- **диагностика:** сообщения об ошибках должны указывать координаты во входном тексте и причину ошибки;
- **расширяемость:** состав встроенных функций допускается сокращать/расширять без изменения ядра сопоставления;
- **переносимость:** предпочтителен язык реализации со сборкой мусора и поддержкой обобщённых типов (в том числе варианты на JVM).

3.8 Обработка ошибок и форматы сообщений

Ошибки целесообразно разделять на классы: лексические (неожиданный символ, незакрытый строковый литерал), синтаксические (неожиданная конструкция, нарушение структуры определения функции) и ошибки выполнения (отсутствие применимого правила). Для повышения качества диагностики в проектировании закладывается единый формат сообщения: координата (строка, столбец), тип ошибки и краткое описание.

3.9 Выбор и сокращение встроенной библиотеки

Сокращённая библиотека встроенных функций проектируется как отдельный слой среды выполнения: каждая встроенная функция имеет строго определённый интерфейс, а таблица примитивов отделена от механизма сопоставления. Такой дизайн позволяет исключать функции, не нужные для рассматриваемого подмножества, не затрагивая ядро выполнения.

3.10 Представление термов и вложенность

В выбранном подмножестве термы образуют минимально необходимый набор для выражения большинства «структурных» преобразований: атомарные тер-

мы (символы-слова, числа, строковые литералы) и скобочные термы. Скобочный терм содержит вложенное выражение, что фактически вводит деревоподобную структуру данных поверх линейной последовательности. С точки зрения проектирования это означает, что контейнер выражения должен хранить не только атомы, но и ссылки на вложенные выражения, причём вложенность может быть произвольной.

Здесь важно различать две роли контейнера:

- на **верхнем уровне** контейнер обслуживает операции над последовательностью термов (концы, конкатенация, разбиение);
- на **уровне скобок** контейнер используется рекурсивно, так как содержащее скобок является самостоятельным выражением.

Таким образом, выбранная структура данных применяется единообразно на всех уровнях вложенности, что упрощает архитектуру и повышает предсказуемость поведения системы.

3.11 Окружение сопоставления и представление подстановок

Результатом сопоставления образца и выражения является окружение (подстановка), связывающая переменные образца со значениями. В проектировании выделяются три класса связываний:

- *s*-переменная связывается с атомарным термом (число, символ-слово или строковый литерал);
- *t*-переменная связывается с термом (включая скобочный);
- *e*-переменная связывается с выражением (цепочкой термов).

С практической точки зрения удобно хранить подстановку в виде отображения «имя переменной → значение», где значение для *e*-переменных является контейнером выражения. Это позволяет строить результат без распаковки цепочек в списки/массивы: результат собирается из фрагментов через операции конкатенации и добавления на концах, опираясь на интерфейс контейнера.

3.12 Контракт механизма сопоставления

Чтобы разделить «логику сопоставления» и «операции над данными», проектирование фиксирует контракт модуля сопоставления:

- вход: образец (последовательность термов с переменными) и выражение-аргумент;
- выход: либо «неуспех», либо подстановка, корректная относительно ограничений типов переменных;
- свойство: если сопоставление успешно, то подстановка совместима со всеми повторными вхождением переменных (одно имя — одно значение).

Такой контракт позволяет в дальнейшем заменять стратегию перебора границ *e*-переменных (например, на более эффективную), не меняя остальную архитектуру. При этом контейнер выражения должен обеспечивать операции, которые нужны сопоставлению: проверку равенства фрагментов, получение префикса/суффикса и сборку новых выражений с разделением структуры.

3.13 Сценарии использования системы

Проектирование исходит из основного сценария — **пакетный запуск**: чтение программы из файла, запуск входной функции с заданным выражением и вывод результата. Такой сценарий опирается на единый формат представления выражений и единый механизм диагностики и соответствует типичному применению инструмента как отдельной программы.

Интерактивный режим (последовательные запуски с разными входными выражениями без повторной загрузки программы) в рамках данного проектирования не фиксируется; его можно рассматривать как возможную доработку или расширение — например, для отладки и исследования поведения правил.

ЗАКЛЮЧЕНИЕ

В НИР сформулирована мотивация использования раскрученных двусторонних очередей для представления объектных выражений в реализации базисного подмножества Рефала-5. Описана структура shallow/deep, инварианты заполненности и причины, по которым достигаются хорошие амортизированные оценки на концах. Выполнено проектирование программной системы и исследовательское обоснование выбора структуры данных для представления объектных выражений.

Проведённое исследование показывает, что выбор контейнера для объектных выражений нельзя рассматривать как второстепенную деталь: в языке переписывания стоимость вычисления во многом определяется тем, насколько эффективно выполняются разбиения и сборки выражений. Подход с раскруткой даёт возможность совместить эффективное версионирование выражений через структурное разделение и хорошие оценки операций на концах, а также создаёт основу для эффективной конкатенации в сценариях интенсивного построения результата.

Сформулированные в работе требования и проектные решения задают основу для дальнейшей доработки реализации: отдельной задачей останется по мере необходимости расширять поддерживаемый синтаксис и библиотеку встроенных функций в соответствии с уточняемыми требованиями к системе и согласованными тестовыми сценариями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. REFAL-5: Programming Guide & Reference Manual [Электронный ресурс]. — URL: <http://refal.botik.ru/book/html/> (дата обр. 28.03.2026).
2. Документация Refal-05 [Электронный ресурс]. — URL: <https://mazdaywik.github.io/Refal-05/> (дата обр. 03.04.2026).
3. Скоробогатов С. Ю. Реализация Рефал-компилятора. Представление данных и язык сборки. — 2008. — URL: <https://bmstu-iu9.github.io/> (дата обр. 05.05.2026) ; Рукопись.
4. Абрамов С. М., Романенко С. А. Представление объектных выражений массивами при реализации языка Рефал : тех. отч. / ИПМ им. М. В. Келдыша АН СССР. — 1988. — № 186.
5. Okasaki C. Purely Functional Data Structures. — Cambridge University Press, 1998.